Test Case 1: Task Creation and Initial Memory Allocation

In this test, we created three tasks using the scheduler and verified that each task was successfully assigned memory by the MMU. Each task received a 64-byte block using our fixed-size allocation strategy. We confirmed that the memory table correctly shows three used blocks and the remaining memory as free. This shows that our memory allocation is working properly and that the scheduler and MMU are connected correctly.

Code Snippet:

```
std::cout << "\n- Test 1: Tasks Creation-\n";
int t1 = sched.create_task("Task1");
int t2 = sched.create_task("Task2");
int t3 = sched.create_task("Task3");
```

Output:

```
- Test 1: Tasks Creation-

--- Process Table ---
Task: Task1 | ID: 1 | State: READY
Task: Task2 | ID: 2 | State: READY
Task: Task3 | ID: 3 | State: READY

 MEMORY TABLE
Status | Handle | Start | End | Size | Current Location | Task-ID
Used   | 0 | 0 | 63 | 64 | 0 | 1
Used   | 1 | 64 | 127 | 64 | 64 | 2
Used   | 2 | 128 | 191 | 64 | 128 | 3
Free   | -1 | 192 | 1023 | 832 | NA | -1

 MEMORY USAGE TABLE
Handle | Base | Limit | Offset | ReadPtr | WritePtr | Active
0 | 0 | 64 | 0 | 0 | 0 | 1
1 | 64 | 64 | 0 | 0 | 0 | 1
2 | 128 | 64 | 0 | 0 | 0 | 1
Mem_Left: 832
Mem_Largest: 832
Mem_Smallest: 832
```

Test Case 2: Memory Write Operation

In this test, we wrote characters (‘A’, ‘B’, ‘C’) into the memory space of Task 1. This shows that once memory is allocated, a task can safely write data into its own memory. We also verified that the write pointer increases correctly after each write, which confirms that our MMU is tracking memory access properly.

Code Snippet:

```
std::cout << "\n- Test 2: Memory Write -\n";

TCB* task = sched.get_task_by_id(t1);
memory.Mem_Write(task->mem_handle, 'A');
memory.Mem_Write(task->mem_handle, 'B');
memory.Mem_Write(task->mem_handle, 'C');

memory.Print_MemInfo();
memory.Mem_Dump(0, 128);
```

Output:

```
- Test 2: Memory Write -

 MEMORY USAGE TABLE
Handle | Base | Limit | Offset | ReadPtr | WritePtr | Active
0 | 0 | 64 | 0 | 0 | 3 | 1
1 | 64 | 64 | 0 | 0 | 0 | 1
2 | 128 | 64 | 0 | 0 | 0 | 1
ABC.............................................................
```

Test Case 3: Memory Read Operation

In this test, we read data from Task 1's memory. The system correctly returned the first character ('A') and updated the read pointer. This confirms that reading works correctly and stays within the allocated memory bounds.

Code Snippet:

```
std::cout << "\n- Test 3: Memory Read -\n";

char ch;
memory.Mem_Read(task->mem_handle, &ch);
std::cout << "Read char: " << ch << std::endl;

memory.Print_MemInfo();
```

Output:

```
- Test 3: Memory Read -
Read char: A

MEMORY USAGE TABLE
Handle | Base | Limit | Offset | ReadPtr | WritePtr | Active
0 | 0 | 64 | 0 | 1 | 3 | 1
1 | 64 | 64 | 0 | 0 | 0 | 1
2 | 128 | 64 | 0 | 0 | 0 | 1
```

Test Case 4: Message Passing (IPC Integration)

In this test, we verified that tasks can communicate using message passing. Task 1 sends a message to Task 2, and Task 2 sends a reply back. We confirmed that each task's mailbox stores the correct messages, including sender, receiver, and message content. This shows that IPC is working together with our system.

Code Snippet:

```
std::cout << "\n- Test 4: Messing Passing -\n";

ipc.Message_Send(t1, t2, "Hello", 1);
ipc.Message_Send(t2, t1, "Reply", 1);

ipc.Message_Print(t1);
ipc.Message_Print(t2);
```

Output:

```
- Test 4: Messing Passing -

Task Number: 1
Message Count: 1
Mail Box:
Source Task-id | Destination Task-id | Message Content | Message Size | Message Type | Message Arrival Time
2 | 1 | Reply | 5 | 1 | 1777058689

Task Number: 2
Message Count: 1
Mail Box:
Source Task-id | Destination Task-id | Message Content | Message Size | Message Type | Message Arrival Time
1 | 2 | Hello | 5 | 1 | 1777058689
```

Test Case 5: Memory Freeing

In this test, we freed the memory allocated to Task 1. The system correctly marked that block as free and replaced its contents with '#' for debugging. We also verified that memory usage updated correctly. This confirms that memory deallocation works and that freed memory is tracked properly.

Code Snippet:

```
std::cout << "\n-Test 5: Memory Freeing -\n";

memory.Mem_Free(task->mem_handle);

memory.Print(&sched);
memory.Print_MemInfo();
Mem_Print_All(memory);
```

Output:

```
-Test 5: Memory Freeing -

 MEMORY TABLE
Status | Handle | Start | End | Size | Current Location | Task-ID
Free   | -1 | 0 | 63 | 64 | NA | -1
Used   | 1 | 64 | 127 | 64 | 64 | 2
Used   | 2 | 128 | 191 | 64 | 128 | 3
Free   | -1 | 192 | 1023 | 832 | NA | -1

 MEMORY USAGE TABLE
Handle | Base | Limit | Offset | ReadPtr | WritePtr | Active
1 | 64 | 64 | 0 | 0 | 0 | 1
2 | 128 | 64 | 0 | 0 | 0 | 1
Mem_Left: 896
Mem_Largest: 832
Mem_Smallest: 64
```

Test Case 6: Memory Coalescing

In this test, we tested memory coalescing. After freeing memory, we combined adjacent free blocks into larger blocks. This helps reduce fragmentation and makes future allocations easier. The output shows that free memory is cleaned up and represented with dots ('.').

Code Snippet:

```
std::cout << "\n- Test 6: Coalesce -\n";

memory.Mem_Coalesce();

memory.Print(&sched);
memory.Mem_Dump(0, 256);
Mem_Print_All(memory);
```

Output:

```
- Test 6: Coalesce -

 MEMORY TABLE
Status | Handle | Start | End | Size | Current Location | Task-ID
Free   | -1 | 0 | 63 | 64 | NA | -1
Used   | 1 | 64 | 127 | 64 | 64 | 2
Used   | 2 | 128 | 191 | 64 | 128 | 3
Free   | -1 | 192 | 1023 | 832 | NA | -1
.................................................................
Mem_Left: 896
Mem_Largest: 832
Mem_Smallest: 64
```

Test Case 7: Allocation Failure

In this test, we tried to allocate more memory than available (5000 bytes). The system correctly returned -1, indicating failure. This confirms that our MMU properly handles edge cases and prevents invalid allocations.

Code Snippet:

```
std::cout << "\n- Test 7: failure-\n";

int fail = memory.Mem_Alloc(5000); // too big

if(fail == -1){
    std::cout << "Allocation failed\n";
}
```

Output:

```
- Test 7: failure-
Allocation failed
```

Final System State Verification

In the final test, we displayed the overall system state, including the scheduler, IPC messages, and memory usage. This confirms that all components (scheduler, IPC, and MMU) are working together correctly as a complete system.

Code Snippet:

```
std::cout << "\n-Final States-\n";

sched.dump(0);
ipc.ipc_Message_Dump();
memory.Print(&sched);
memory.Print_MemInfo();
Mem_Print_All(memory);

return 0;
```

Output:

```
-Final States-

--- Process Table ---
Task: Task1 | ID: 1 | State: READY
Task: Task2 | ID: 2 | State: READY
Task: Task3 | ID: 3 | State: READY

========== IPC SYSTEM DUMP ==========
Task 1 | Messages: 1
Task 2 | Messages: 1
Task 3 | Messages: 0
=====================================

 MEMORY TABLE
Status | Handle | Start | End | Size | Current Location | Task-ID
Free   | -1 | 0 | 63 | 64 | NA | -1
Used   | 1 | 64 | 127 | 64 | 64 | 2
Used   | 2 | 128 | 191 | 64 | 128 | 3
Free   | -1 | 192 | 1023 | 832 | NA | -1

 MEMORY USAGE TABLE
Handle | Base | Limit | Offset | ReadPtr | WritePtr | Active
1 | 64 | 64 | 0 | 0 | 0 | 1
2 | 128 | 64 | 0 | 0 | 0 | 1
Mem_Left: 896
Mem_Largest: 832
Mem_Smallest: 64
```